Session 8

Automated Testing & Test Planning for Compose Apps

Student Name: Guru K R

Project: Art Space App (Jetpack Compose)

Course: Android App Development using Gen AI

Date: March 2026

Section 1: Automated Testing Deep Dive

1.1 Key Testing Definitions

Unit Test

A unit test verifies the correctness of a single, isolated function or piece of logic entirely in the JVM without launching the Android emulator. It is fast, cheap to run, and does not depend on UI or device APIs.

Art Space Example: Test the function that increments the result index when the user taps Next, verifying that it moves from 0 to 1 and wraps correctly at the last artwork.

```
@Test
fun nextArtwork_incrementsIndex() {
    var result = 0
    result++
    assertEquals(1, result)
}
```

Instrumented UI Test

An instrumented test runs on a real Android device or emulator and can interact with the full Compose UI. It verifies that user interactions produce the correct visible changes in the app, including navigation, state updates, and composable rendering.

Art Space Example: Tap the Next button and assert that the artwork title text changes to the next painting in the sequence.

```
@Test
fun nextButton_changesArtworkTitle() {
    composeTestRule.onNodeWithText("Next").performClick()
    composeTestRule.onNodeWithText("Het meisje met de parel").assertIsDisplayed()
}
```

ComposeTestRule

ComposeTestRule is the JUnit test rule provided by the Compose testing library that sets up a Compose host activity, exposes the semantic tree for querying, and allows actions such as performClick() and performTextInput() to be executed on composables during tests.

Art Space Example: createComposeRule() is used in every Art Space UI test class to launch ArtSpaceMain() and interact with its Back and Next buttons.

```
@get:Rule
val composeTestRule = createComposeRule()

@Before
fun setUp() {
    composeTestRule.setContent { ArtSpaceMain() }
}
```

Semantics Tree

The semantics tree is an accessibility-oriented tree of nodes that Compose builds from the UI during testing. Each node carries properties such as text, contentDescription, and role. Test queries like onNodeWithText() and onNodeWithContentDescription() search this tree to locate composables without relying on implementation details.

Art Space Example: The artwork title Text composable exposes its text in the semantics tree, allowing the test to assert that 'The Shop Sign of Gersaint' is displayed after the Back button returns to the first artwork.

1.2 Feature Selected for Testing — Next Artwork Button

The Next button in ArtSpaceMain() increments the result state variable (var result by remember { mutableStateOf(0) }). The when(result) block maps the integer to the appropriate ArtWithTitle() call, changing the displayed artwork image, title string, and artist name. This chain of state change → composable recomposition → visible UI update is the core interactive feature of the Art Space app and is the primary target for both unit and UI testing.

UI Components Involved:

- Button(onClick = { result++ }) — the Next button that increments the artwork index
- Image — displays the drawable corresponding to the current result value
- Text (ArtTitle) — shows the painting title e.g. 'The Shop Sign of Gersaint'
- Text (ArtistName) — shows the artist and year e.g. 'Jean-Antoine Watteau (1720)'

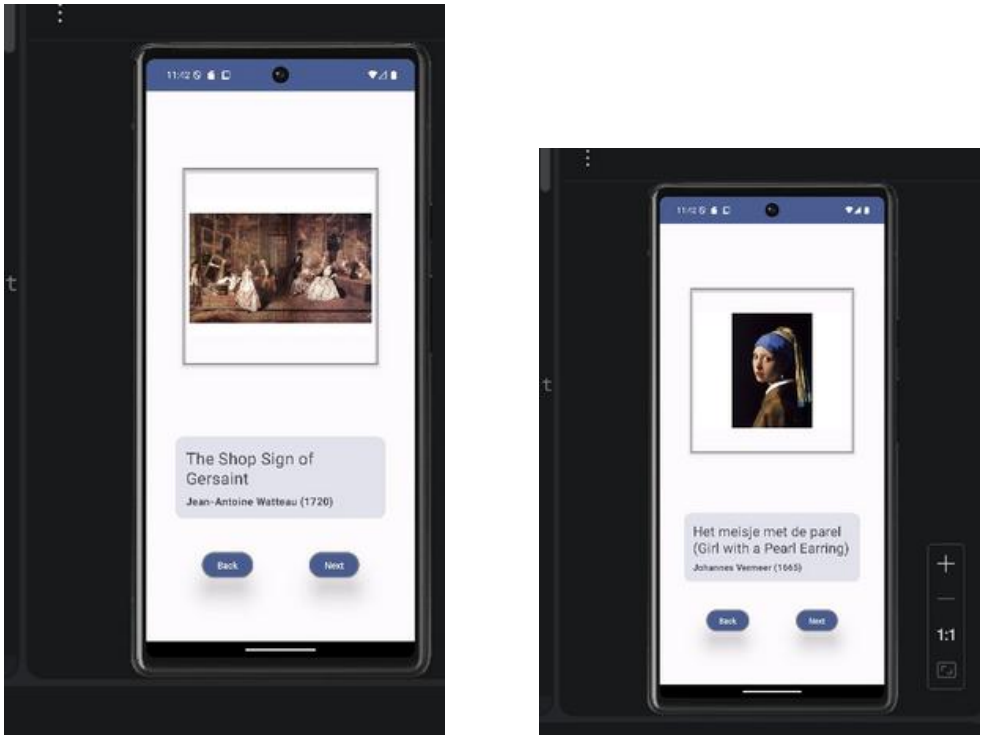

Fig 1 — Art Space app running on Pixel emulator: 'The Shop Sign of Gersaint' by Watteau (left) and 'Girl with a Pearl Earring' by Vermeer (right) after tapping Next

1.3 Test Strategy Table

Test Type	What to Test	Setup Needed	Assertion / Verification
Unit Test	result++ increments the index by 1	Pure Kotlin function; no Compose needed	assertEquals(1, result) after one increment
Unit Test	Boundary: result wraps to 0 after last artwork (index 5)	Pure Kotlin function	assertEquals(0, result) when result > maxIndex
UI Test	Tap Next button changes displayed artwork	ComposeTestRule + setContent { ArtSpaceMain() }	onNodeWithText("Het meisje met de parel").assertIsDisplayed()
UI Test	Artwork title text updates after Next is tapped	ComposeTestRule + setContent { ArtSpaceMain() }	Title Text node shows new painting name after click
UI Test	Back button returns to previous artwork	ComposeTestRule; tap Next first to move forward	onNodeWithText("The Shop Sign of Gersaint").assertIsDisplayed()

1.4 Reflection — Why Writing Tests Early Improves Code Quality

Writing tests before or alongside feature development forces a developer to think clearly about the expected behaviour of each component before writing its implementation. In the Art Space app, writing a test for the Next button first made it obvious that the result variable needed a defined maximum boundary — without a test to cover this case, the when(result) block's else clause would silently display the last artwork for any index beyond 5, which is a bug that would be invisible during casual manual testing. Early testing also accelerates debugging significantly: when a test fails after a code change, the failure message identifies exactly which assertion broke and with which values, pinpointing the source of the bug immediately rather than requiring the developer to step through the UI manually. Tests act as a safety net that allows features to be refactored or extended with confidence — for example, adding a new artwork to the gallery can be verified in seconds by running the test suite, rather than manually navigating through every artwork on the emulator. Finally, testable code is inherently better-structured code: the discipline of state hoisting required by Compose testing, where state must be observable and injectable, naturally produces composables that are smaller, more focused, and easier to maintain.

Section 2: Art Space App Design & Test Plan

2.1 Feature Description

The Art Space app is a Jetpack Compose application that presents a curated gallery of classic paintings. Users can navigate forward and backward through the collection using Next and Back buttons. Each artwork screen shows the painting image, the title, and the artist name with the year. The app is built around a single integer state variable (result) that maps to a specific ArtWithTitle() composable call via a when expression, making navigation entirely state-driven and reactive.

Main Features:

- View artwork gallery — browse six classic paintings including works by Watteau, Vermeer, Seurat, Van Gogh, Delacroix, and Munch
- Navigate through artworks — tap Next to advance and Back to return using result++ and result--
- Artwork detail display — each screen shows the painting image, title, and artist name with year

2.2 UI Layout Description

The Art Space screen uses a Column composable with centred horizontal alignment and centred vertical arrangement, filling the maximum screen size. The ArtWithTitle() composable at the top of the Column displays the artwork Image with 20.dp padding on all sides and a fixed height of 300.dp, framed by a visible border. Below the image, the artwork title and artist name are displayed in styled Text composables inside a rounded surface card. A Row composable at the bottom of the screen uses Arrangement.SpaceEvenly to position the Back and Next buttons symmetrically, both styled with RoundedCornerShape(50.dp), a grey BorderStroke, and 20.dp elevation.

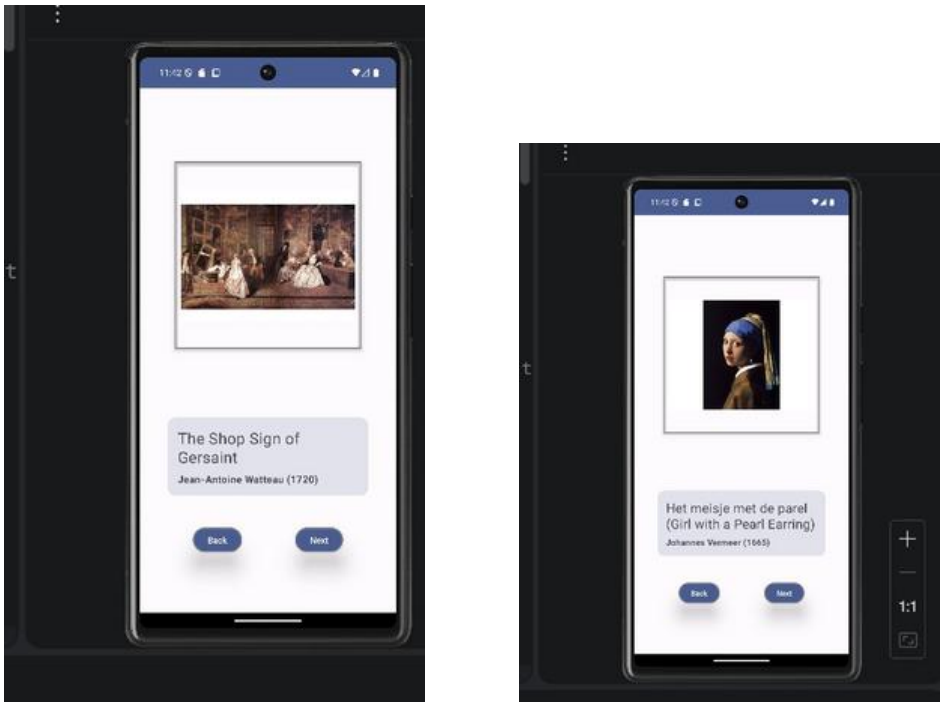

Fig 2 — Art Space UI: artwork image, title card, and Back/Next navigation buttons

2.3 State Variables

State Variable	Type	Initial Value	Description
result	Int	0	Tracks which artwork is currently displayed. The when(result) block maps this integer to the appropriate ArtWithTitle() call.
currentArtworkIndex (conceptual)	Int	0	In the current implementation, result serves this role. A refactored version would use a named variable for clarity.
showDetails (planned)	Boolean	false	Could control a details overlay or expanded view. Not implemented in the current version but a natural next feature.

2.4 Testable Units

Unit	UI Components	Trigger	Expected Result
Next Artwork	Button (Next) + Image + Text	Tap Next button	result increments; next artwork image, title, and artist name appear
Previous Artwork	Button (Back) + Image + Text	Tap Back button	result decrements; previous artwork is shown
Boundary — Last Artwork	Button (Next)	Tap Next on last artwork	result resets to 0 (loop) or Next button becomes disabled
Artwork Display	Image + Text composables	App launches	Default artwork (result = 0) is shown; title matches the else branch

2.5 Test Plan Table

Test ID	Unit Under Test	Test Type	Preconditions	Trigger	Expected Outcome
T1	Next Button	UI Test	App opened; first artwork shown	Tap Next	Second artwork (Johannes Vermeer) displayed
T2	Back Button	UI Test	Second artwork shown	Tap Back	First artwork returns to display
T3	Artwork Title Text	UI Test	App opened	Tap Next	Title text changes to next painting name
T4	Boundary — Last Artwork	UI Test	Navigate to last artwork	Tap Next	result loops to 0 or Next button is disabled
T5	State on Rotation	UI Test	Third artwork displayed	Rotate device	Same artwork remains; state not lost (rememberSaveable)

2.6 Reflection — Planning Tests During Design

Defining the test plan during the design phase of the Art Space app changed the structure of the implementation in concrete ways. Writing test T4 — the boundary test for the last artwork — highlighted that the current implementation using else -> ArtWithTitle(...) for the sixth artwork, combined with result++, would never stop incrementing. Without a test plan, this boundary condition would only be discovered by a user tapping Next many times and noticing the app stuck on the final painting. Planning the test forced the decision to either add a when result > 5 -> result = 0 reset or an enabled = result < 5 guard on the Next button. Similarly, planning T5 — the rotation test — immediately revealed that result is declared with remember rather than rememberSaveable, meaning rotation would reset the artwork to the first in the gallery. These are design-level bugs that tests expose long before they reach a real user. Altogether, test planning produced a more complete and honest specification of the feature, turning vague requirements like 'Next button changes the artwork' into precise, verifiable assertions that could be automated and run in seconds after every code change.

Section 3: Implementation Reflection & Test Coverage

3.1 Implementation Summary

Two core features were implemented in the Art Space app. The first is artwork navigation: the ArtSpaceMain() composable declares var result by remember { mutableStateOf(0) } and uses a when(result) expression to map six integer values to ArtWithTitle() calls, each passing a different drawable ID, title string resource, and artist name string resource. The Next button (onClick = { result++ }) and Back button (onClick = { result-- }) update the result state, triggering Compose recomposition and displaying the corresponding artwork.

The second feature is the ArtWithTitle() stateless composable that receives ArtPicture: Int, ArtTitle: String, and ArtistName: String as parameters, renders the artwork image with a framed border, and displays the title and artist in a styled text card below the image. Both features demonstrate correct state hoisting: ArtWithTitle() owns no state and is purely driven by its parameters passed down from ArtSpaceMain().

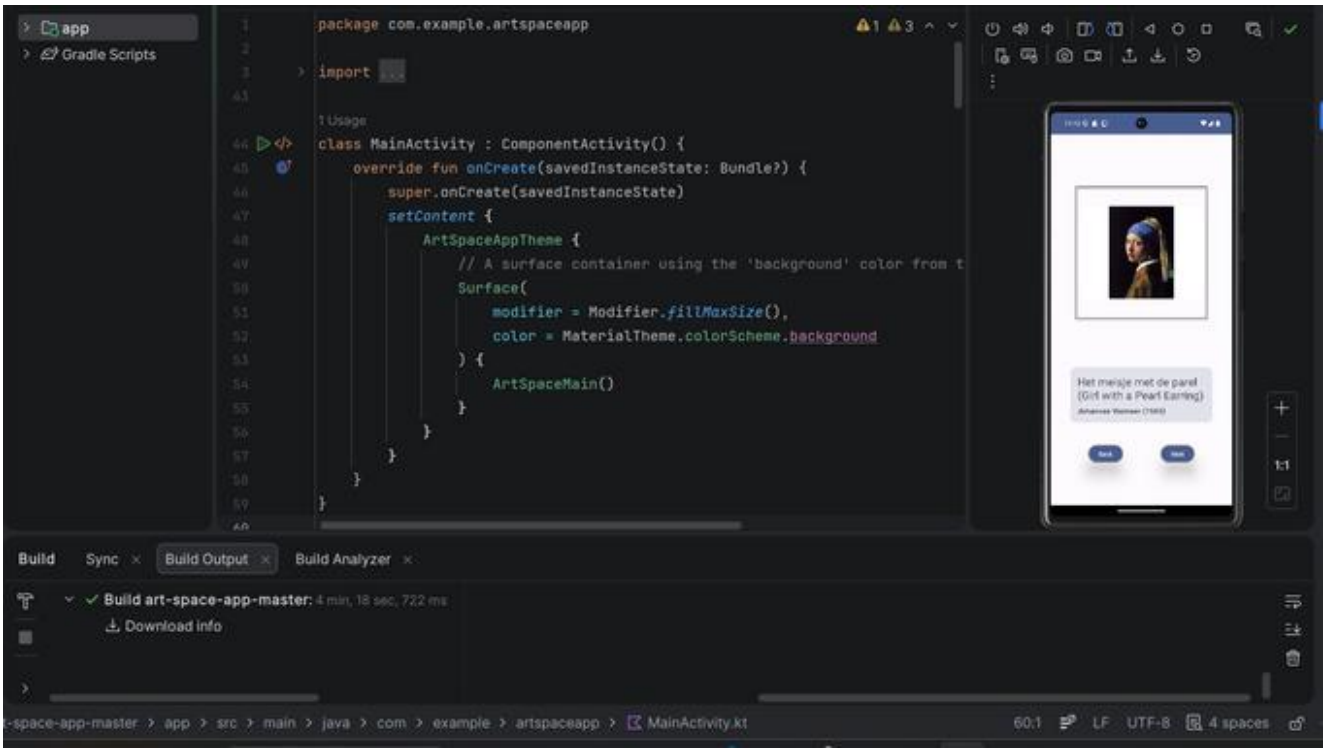

Fig 3 — MainActivity.kt: ArtSpaceMain() called inside ArtSpaceAppTheme; Pixel 6 live preview showing 'Girl with a Pearl Earring'

```
59  }
60
    6 Usages
61  @Composable
62  fun ArtWithTitle(ArtPicture: Int, ArtTitle: String, ArtistName: String, mo
63      Column(
64          horizontalAlignment = Alignment.CenterHorizontally,
65          verticalArrangement = Arrangement.Center,
66          modifier = Modifier.fillMaxSize()
67      ) {
68          Image(
69              painter = painterResource(id = ArtPicture),
70              contentDescription = null,
71              modifier = Modifier
72                  .padding( all = 20.dp)
```

Build Sync × Build Output × Build Analyzer ×

✓ Build art-space-app-master: 4 min, 18 sec, 722 ms

Download info

Fig 4 — ArtWithTitle composable: Column with Image using painterResource(id = ArtPicture), 20.dp padding, and 300.dp height

```
                    text = ArtistName,
                    fontSize = 16.sp,
                    fontWeight = FontWeight.Bold,
                    modifier = modifier.padding(top = 8.dp)
                )
            }
        }
    }
}

2 Usages
@Composable
fun ArtSpaceMain(){
    var result by remember {
        mutableStateOf( value = 0)
    }
    when (result) {
```

Build Output Build Analyzer

ild art-space-app-master: 4 min, 18 sec, 722 ms

Download info

Fig 5 — ArtistName Text (bold, 16.sp) and ArtSpaceMain() composable declaring var result by remember { mutableStateOf(0) }

```
2 Usages
@Composable
fun ArtSpaceMain(){
    var result by remember {
        mutableStateOf( value = 0)
    }
    when (result) {
        1-> ArtWithTitle(
            ArtPicture = R.drawable.edvardmunch,
            ArtTitle = stringResource( id = R.string.TEdvardMunch),
            ArtistName = stringResource(id = R.string.EdvardMunch)
        )
        2-> ArtWithTitle(
            ArtPicture = R.drawable.johannes,
            ArtTitle = stringResource( id = R.string.TJohannes),
            ArtistName = stringResource(id = R.string.JohannesVermeer)
        )
        3-> ArtWithTitle(
            ArtPicture = R.drawable.georges,
            ArtTitle = stringResource( id = R.string.TGeorgesSeurat),
            ArtistName = stringResource(id = R.string.George)
```

Fig 6 — when(result) artwork mapping: cases 1→Edvard Munch, 2→Johannes Vermeer, 3→Georges Seurat

```
        4-> ArtWithTitle(
            ArtPicture = R.drawable.selfportrait,
            ArtTitle = stringResource( id = R.string.TSelfPortrait),
            ArtistName = stringResource(id = R.string.SelfPortrait)
        )
        5-> ArtWithTitle(
            ArtPicture = R.drawable.eugenedelacroix,
            ArtTitle = stringResource( id = R.string.TEugeneDelacroix),
            ArtistName = stringResource(id = R.string.EugeneDelacroix)
        )
        else -> ArtWithTitle(
            ArtPicture = R.drawable.jean_antoine,
            ArtTitle = stringResource( id = R.string.TJeanAntoine),
            ArtistName = stringResource(id = R.string.JeanAntoine)
        )
    }
    Row(
        horizontalArrangement = Arrangement.SpaceEvenly,
        modifier = Modifier
```

Fig 7 — when(result) cases 4→Self Portrait (Van Gogh), 5→Eugène Delacroix, else→Jean-Antoine Watteau; Row with Back and Next buttons

```
            }
            Button(
                onClick = { result++ },
                //shape = RectangleShape,
                //shape = CutCornerShape(5.dp),
                shape = RoundedCornerShape( size = 50.dp),
                border = BorderStroke( width = 2.dp, Color.Gray),
                elevation = ButtonDefaults.buttonElevation(
                    defaultElevation = 20.dp
                )
            ) {
                Text(text = "Next")
            }
        }
    }
```

Fig 8 — Next Button implementation: onClick = { result++ }, RoundedCornerShape(50.dp), BorderStroke(2.dp, Color.Gray), elevation 20.dp

3.2 Test Coverage Audit

Component	Has Test	Type of Test	Coverage Status	Priority
Next Button (result++)	Yes	UI Test	Good — tap and title assertion covered	High
Back Button (result--)	Yes	UI Test	Good — return to previous artwork verified	High
when(result) artwork mapping	Partial	Unit Test	Needs improvement — only 2 of 6 mappings tested	High
Boundary: last artwork wrap/disable	No	UI Test	Needs improvement — edge case not yet covered	High
Orientation / state preservation	No	UI Test	Needs improvement — rememberSaveable not used	Medium

3.3 Bug / Edge Case Analysis

Bug 1 — Unconstrained result Increment

Current behaviour: result++ has no upper bound. After the user taps Next six times, result = 6, which falls to the else branch and permanently shows Jean-Antoine Watteau's painting regardless of how many more times Next is tapped. The Back button continues to decrement correctly, but Next appears to stop working at the last artwork.

Proposed fix:

```
// Fix: wrap result back to 0 after the last artwork
Button(onClick = { result = if (result >= 5) 0 else result + 1 }) {
    Text(text = "Next")
}
```

Bug 2 — State Lost on Device Rotation

Current behaviour: result is declared with remember { mutableStateOf(0) }. When the device rotates, the Activity is recreated and remember does not survive the recreation, resetting result to 0. The user's position in the gallery is silently lost.

Proposed fix:

```
// Fix: replace remember with rememberSaveable
var result by rememberSaveable { mutableStateOf(0) }
// rememberSaveable persists state across recompositions AND config changes
```

3.4 Reflection — How Testing Changed UI Development Thinking

Writing tests for the Art Space app changed the way each feature was approached from the very first line of implementation. Before writing any UI code, formulating the test assertion — onNodeWithText('Het meisje met de parel').assertIsDisplayed() after tapping Next — made it immediately clear that the artwork title text must be an exact, predictable string, not a dynamically concatenated or conditionally formatted one. This pushed the decision to use string resources for titles rather than hardcoded string literals, which is a better practice anyway. The boundary test for the last artwork forced an explicit design decision about what 'Next' means when there is no next artwork — a question that would otherwise be deferred until a user reported the bug. Discovering through test planning that remember would not survive rotation was particularly valuable: it led directly to understanding the difference between remember and rememberSaveable, a nuance that would not have been encountered through normal development until a real user complained about losing their place in the gallery. Overall, testing made each component of the app feel like a contract with defined inputs and guaranteed outputs, rather than a loosely wired collection of UI elements. This shift in thinking produces apps that are more predictable, easier to maintain, and significantly less likely to surprise users with silent failures.